public void join() throws InterruptedException

The main purpose of join() method to put the current thread into temporarily waiting state until the other thread finish its execution.

Here the currently executing thread stops its execution and the thread goes into the waiting state. The current thread remains in the wait state until the thread on which join() method is invoked has achieved its dead state.

It also throws checked exception i.e InterruptedException so better to use try catch or declare the method as throws.

It is a non static method so we can call this method with the help of Thread object reference.

//Programs :

```
package com.ravi.join;

class Even extends Thread
{
    @Override
    public void run()
    {
      IO.println("Even thread started");
      String name = Thread.currentThread().getName();

      //print all the even numbers from 1 to 10
      for(int i=2; i<=10; i=i+2)
      {
          IO.println("Even number "+i+" by "+name+" thread");

          try
          {
              Thread.sleep(1000);
          }
          catch(InterruptedException e)
          {

          }
      }
       IO.println("Even thread ended");
    }
}

class Odd extends Thread
{
@Override
public void run()
{
    IO.println("Odd thread started");
    String name = Thread.currentThread().getName();

    //print all the odd numbers from 1 to 100
    for(int i=1; i<=10; i=i+2)
    {
        IO.println("Odd number "+i+" by "+name);
        try
        {
            Thread.sleep(1000);
        }
        catch(InterruptedException e)
        {

        }

    }
    IO.println("Odd thread ended");
}
}

public class JoinDemo1
{
    public static void main(String[] args)
    {
        IO.println("Main thread started!!!!");

        Even even = new Even();
        Odd odd = new Odd();
        even.setName("even");
        odd.setName("Odd");

        even.start();

        try
        {
        even.join();  //Main thread is waiting for even thread to complete
        }
        catch(InterruptedException e)
        {
            e.printStackTrace();
        }
        IO.println("Main thread wake up");

        odd.start();

        IO.println("Main thread ended!!!!");


    }
}
```

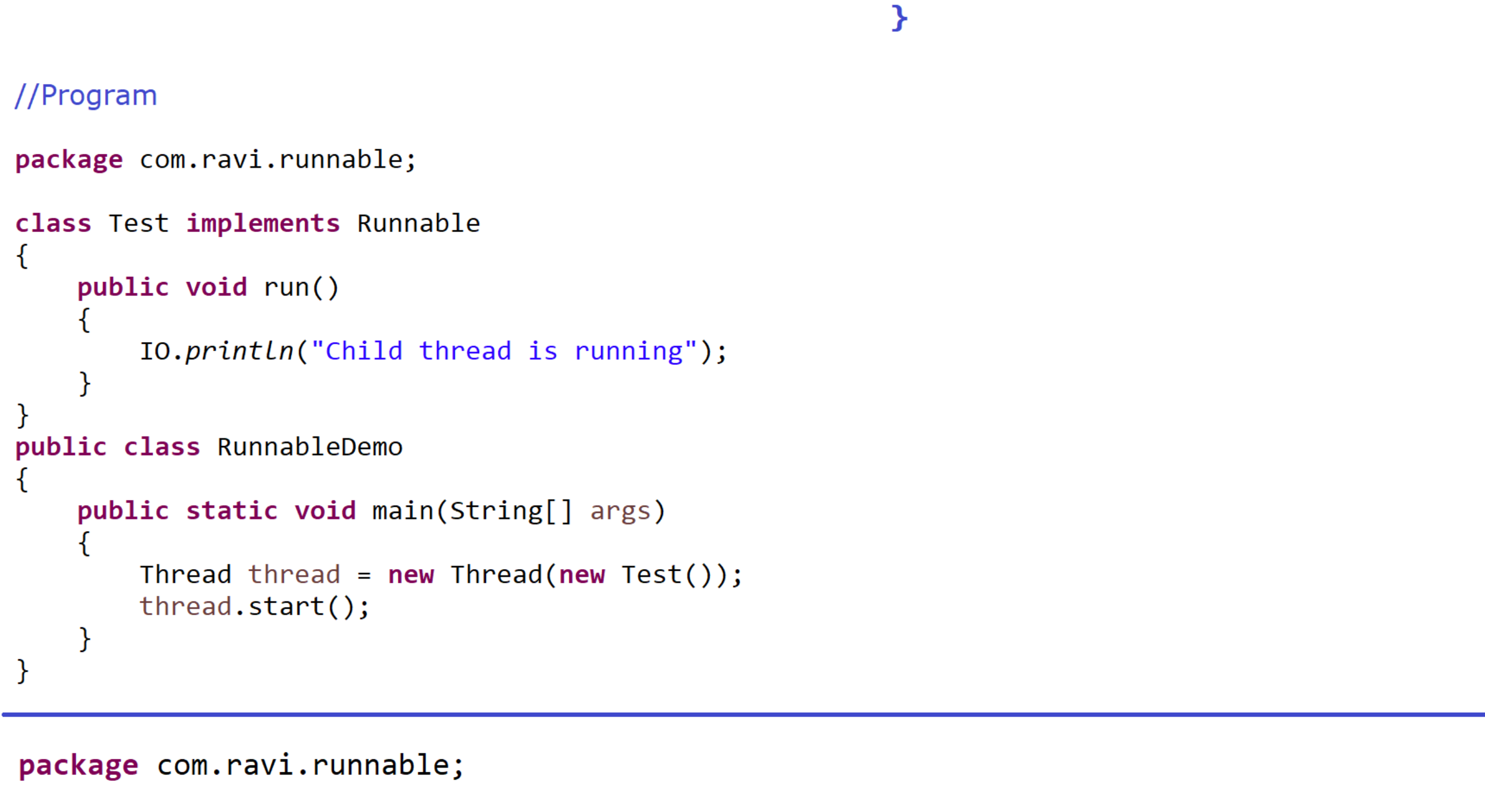
```
package com.ravi.join;

//Main Thread -> Alpha thread -> Beta Thread


class Alpha extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println(name+ " thread started!!!");

        Beta beta = new Beta();
        beta.setName("Beta");
        beta.start();

        try
        {
            beta.join();
        }
        catch(InterruptedException e)
        {

        }
        IO.println(name+" thread re-started!!!");

        for(int i=1; i<=10; i++)
        {
            IO.println(i+" by "+name+" thread");

            try
            {
                Thread.sleep(500);
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
        }
        IO.println(name+ " thread ended!!!");

    }
}

class Beta extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println(name+ " thread started!!!!");

        for(int i=1; i<=50;i++)
        {
            IO.println(i+" by "+name+" thread");
        }

        IO.println(name+ " thread ended!!!!");

    }
}

public class JoinDemo2
{
    public static void main(String[] args)
    {
        IO.println("Main thread started");

        Alpha alpha = new Alpha();
        alpha.setName("Alpha");
        alpha.start();
        try
        {
          alpha.join();
        }
        catch (InterruptedException e)
        {
          e.printStackTrace();
        }
        IO.println("Main thread ended");
    }
}
```

```
package com.ravi.join;

public class JoinDemo3
{
    public static void main(String[] args) throws InterruptedException
    {
        IO.println("Main thread started");

        Thread t = Thread.currentThread();

        for(int i=1; i<=10; i++)
        {
            IO.println(i+" by "+t.getName()+" thread ");
            Thread.sleep(1000);
        }

        t.join();  //Main thread is waiting for main thread to complete [Deadlock]

        IO.println("Main thread ended");


    }
}
```

```
package com.ravi.join;
//Online banking amount transfer after OTP Verification

class BalanceCheck extends Thread
{
    @Override
    public void run()
    {
        IO.println("Balance Verification is in progress!!!");
        try
        {
            Thread.sleep(1000);
        }
        catch(InterruptedException e)
        {
            e.printStackTrace();
        }
        IO.println("Sufficient Balance is available !!!");
    }
}

class OTPVerification extends Thread
{
    @Override
    public void run()
    {
        IO.println("OTP Verification is in progress!!!");
        try
        {
            Thread.sleep(5000);
        }
        catch(InterruptedException e)
        {
            e.printStackTrace();
        }
        IO.println("OTP Verified!!!");
    }
}


public class JoinDemo4
{
    public static void main(String[] args) throws InterruptedException
    {
        //Main thread wants to transfer some amount

        IO.println("Amount transfer process initiated by main thread....");

        BalanceCheck bal = new BalanceCheck();
        OTPVerification otp = new OTPVerification();

        bal.start();
        otp.start();

        bal.join();
        otp.join();

        IO.println("Amount transfer completed");

    }
}
```

Assigning the target for the thread by using Runnable interface :

How we can execute out own run() method in implements Runnable approach :



```
class Test implements Runnable
{
    @Override
    public void run()
    {
        IO.println("Child thread is running");
    }
}
public class RunnableDemo
{
    public static void main(String[] args)
    {
        Thread thread = new Thread(new Test());
        thread.start();

    }
}
```

```
class Thread implements Runnable
{
    private Runnable target;

    public Thread(Runnable target)
    {
        this.target = target;
    }

    public void start()
    {
        this.run();
    }

    public void run()
    {
        if(target!=null)
        {
            target.run();
        }
}
```

//Program

```
package com.ravi.runnable;

class Test implements Runnable
{
    public void run()
    {
        IO.println("Child thread is running");
    }
}
public class RunnableDemo
{
    public static void main(String[] args)
    {
        Thread thread = new Thread(new Test());
        thread.start();
    }
}
```

```
package com.ravi.runnable;

class Tatkal implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println("Congratulations! "+name+" you ticket is booked under Tatkal Scheme");
    }
}
class PremimumTatkal implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println("Congratulations! "+name+" you ticket is booked under PremimumTatkal
Scheme");
    }
}

public class AssigningTarget {

    public static void main(String[] args)
    {
        Thread scott = new Thread(new Tatkal()," Scott");
        Thread alen = new Thread(new PremimumTatkal(),"Alen");

        scott.start();
        alen.start();

    }

}
```